Project Testing

Project Title

Implement Client Script & UI Policy (Incident)

1. Introduction

The Project Testing phase verifies whether the implemented UI Policies and Client Scripts work correctly on the ServiceNow Incident table. The testing focuses on mandatory field enforcement, automatic value updates, form validation, reverse conditions, and restrictions on list editing.

2. Testing Objectives

The main objectives of testing are:

- To verify the High Impact UI Policy.
- To verify the Urgency field behavior.
- To verify automatic Urgency setting.
- To verify Assigned To validation.
- To verify the restriction on State list editing.
- To verify that valid Incident records can be saved successfully.
- To verify that the reverse condition works correctly.

3. Test Case 1 – Mandatory Enforcement

Test Condition

- Set **Impact = High**.
- Leave **Assigned To** empty.
- Try to submit the Incident.

Expected Result

The Incident should **not be saved**. An error message should be displayed indicating that Assigned To is mandatory for High Impact incidents.

The UI Policy actions should also be applied.

4. Test Case 2 – Successful Save

Test Condition

- Set **Impact = High**.
- Fill the **Assigned To** field.

- Submit the Incident.

Expected Result

The Incident should be saved successfully.

The following behaviors are also verified:

- Urgency is automatically set to High.
- Configured read-only behavior works correctly.
- State restrictions are maintained.

5. Test Case 3 – Reverse Condition

Test Condition

- Initially set **Impact = High**.
- Change Impact from **High to Medium**.

Expected Result

When the condition becomes false:

- Assigned To should no longer be mandatory.
- Urgency should become editable.
- The Incident should be saved successfully.

This verifies the **Reverse if false** behavior of the UI Policy.

6. Test Case 4 – List Edit Blocking

Test Condition

- Open the Incident list.
- Try to edit the **State** field directly using list editing.

Expected Result

An alert should be displayed:

“State cannot be updated using list editing. Please open the Incident.”

The State value should remain unchanged.

7. Test Case 5 – Form-Based State Update

Test Condition

- Open an Incident record.
- Change the **State** field through the Incident form.
- Save the record.

Expected Result

The State change should be saved successfully through the form. The restriction applies specifically to list editing.

8. Testing Summary

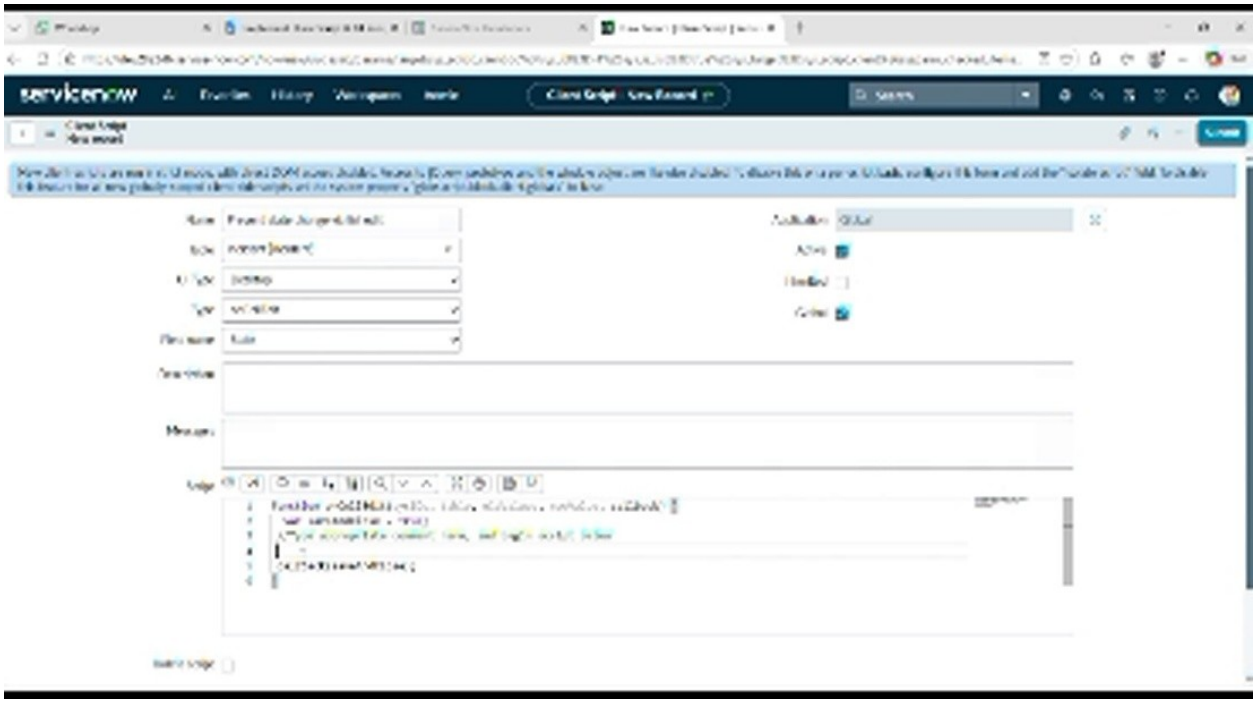
Test Case	Condition	Expected Result
TC01	High Impact + Assigned To empty	Record not saved
TC02	High Impact + Assigned To filled	Record saved successfully
TC03	High → Medium Impact	Mandatory/read-only behavior reversed
TC04	State changed through list edit	Change blocked
TC05	State changed through form	Change saved successfully

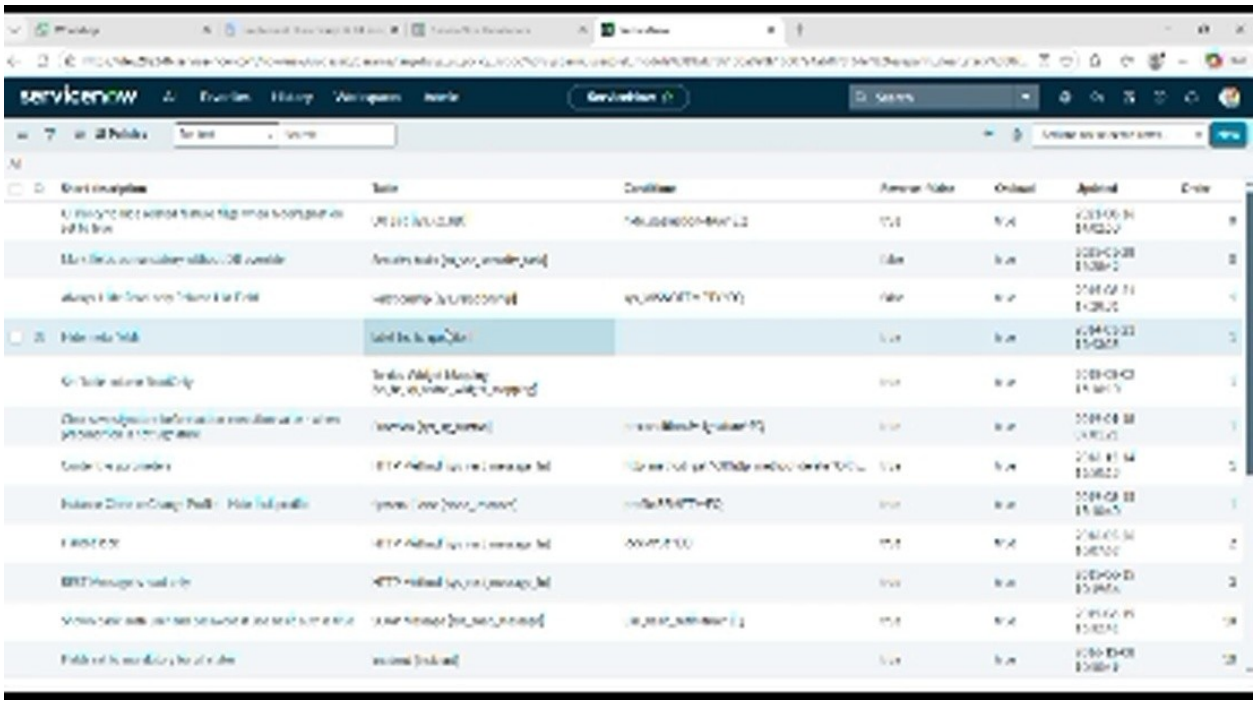
9. Validation of Client Scripts

The testing verifies all three Client Script types used in the project:

- **onChange:** Automatically sets Urgency when Impact becomes High.
- **onSubmit:** Prevents saving when Assigned To is missing for High Impact.
- **onCellEdit:** Prevents State changes through list editing.

Screenshot





10. Conclusion

The Project Testing phase verifies the major functional requirements of the Incident configuration. The tests cover mandatory validation, successful record submission, automatic Urgency updates, reverse UI Policy behavior, and State list-edit restrictions. These tests confirm that the configured UI Policies and Client Scripts perform the intended Incident form controls.